

Math Foundations of Machine Learning

Bias and Variance

Gabriel Wendell Celestino Rocha

July 2025

Table of Contents

Bias and Variance

- Problem Setup

- Bias and Variance Tradeoff

- Model Complexity

Principal Components Analysis

- Problem Setting

- Maximum Variance Perspective



Bias and Variance

Problem Setup

Let $\mathbf{x} \in \mathcal{X} \subset \mathbb{R}^D$ be the input features, and $y : \mathbb{R}^D \rightarrow \mathbb{R}$ be the target variable.

- The true relationship is $y_n = f(\mathbf{x}_n) + \varepsilon$, where:
 - $f(\mathbf{x}_n)$ is the **true function**
 - ε is the noise with $\mathbb{E}[\varepsilon] = 0$ and $\mathbb{V}(\varepsilon) = \sigma^2$.
- $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ is the training dataset drawn i.i.d. from the data distribution $\mathcal{P}(\mathbf{x}, y)$.
- Let $\hat{f}(\mathbf{x}; \mathcal{D})$ be a learned model based on data $\mathcal{D}$.

We are interested in the **expected prediction error** at a point $\mathbf{x}$ averaged over all possible training sets and noise realizations.

Mean Squared Error (MSE)

The **expected squared error** of our model prediction at a point $\mathbf{x}$ is:

$$\mathbb{E}_{\mathcal{D}, \varepsilon} \left[\left(\hat{f}(\mathbf{x}_n; \mathcal{D}) - y_n \right)^2 \right] .$$

Using $y_n = f(\mathbf{x}_n) + \varepsilon$, we can rewrite this as:

$$\mathbb{E}_{\mathcal{D}, \varepsilon} \left[\left(\hat{f}(\mathbf{x}_n; \mathcal{D}) - f(\mathbf{x}_n) - \varepsilon \right)^2 \right] .$$

Expanding the square and using the independence of $\hat{f}$ and ε we get:

$$\mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}(\mathbf{x}_n; \mathcal{D}) - f(\mathbf{x}_n) \right)^2 \right] + \underbrace{\mathbb{E} [\varepsilon^2]}_{\sigma^2} .$$

Bias-Variance Decomposition

Let $\hat{f}(\mathbf{x}) := \mathbb{E}_{\mathcal{D}} [\hat{f}(\mathbf{x}; \mathcal{D})]$ be the expected prediction. Then:

$$\mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}(\mathbf{x}_n; \mathcal{D}) - f(\mathbf{x}_n) \right)^2 \right] = \underbrace{\left[\tilde{f}(\mathbf{x}_n) - f(\mathbf{x}_n) \right]^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}(\mathbf{x}_n; \mathcal{D}) - \tilde{f}(\mathbf{x}_n) \right)^2 \right]}_{\text{Variance}}$$

$$\therefore \mathbb{E} \left[\left(\hat{f}(\mathbf{x}_n) - y_n \right)^2 \right] = \underbrace{\left[\mathbb{E} [\hat{f}(\mathbf{x}_n)] - f(\mathbf{x}_n) \right]^2}_{\text{Bias}^2} + \underbrace{\mathbb{E} \left[\left(\hat{f}(\mathbf{x}_n) - \mathbb{E} [\hat{f}(\mathbf{x}_n)] \right)^2 \right]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible Noise}}$$

Bias and Variance

Bias and Variance

- **Bias:** *an error introduced by very strong model assumptions.*

A very simple model (e.g., linear regression for highly nonlinear data) has high bias. As a consequence, it fails to capture the complexity of the data $\implies$ **underfitting**.

- **Variance:** *It is how much the model fluctuates with small changes in the training data.*

A very complex model (e.g., a 20th-degree polynomial) tends to memorize the training data. As a consequence, it has high variance and low generalization ability $\implies$ **overfitting**.

Bias-Variance Trade-off

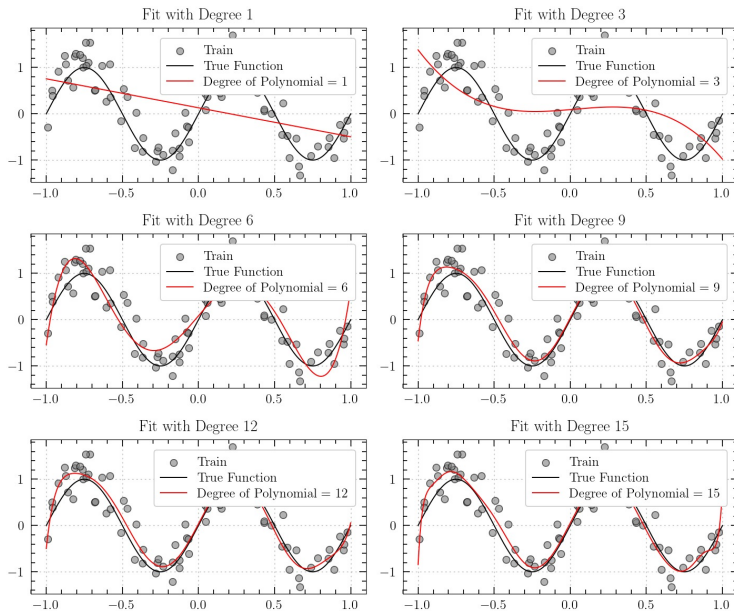
Summary

- $\uparrow$ model complexity $\implies \downarrow$ bias
- $\uparrow$ model complexity $\implies \uparrow$ variance

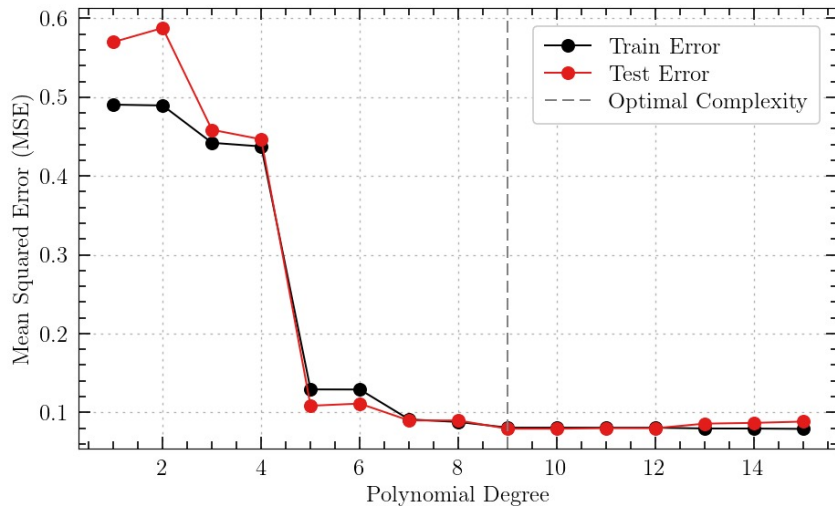
Examples

1. **Linear Regression:** High bias & Low variance $\implies$ Too simple for curved data
2. **Random Forest (very deep):** Low bias & High variance $\implies$ Can memorize noise
3. **Shallow Neural Network:** Medium bias & Medium variance $\implies$ Good balance point
4. **Very deep neural network without regularization:** Low bias & High variance $\implies$ Risk of overfitting

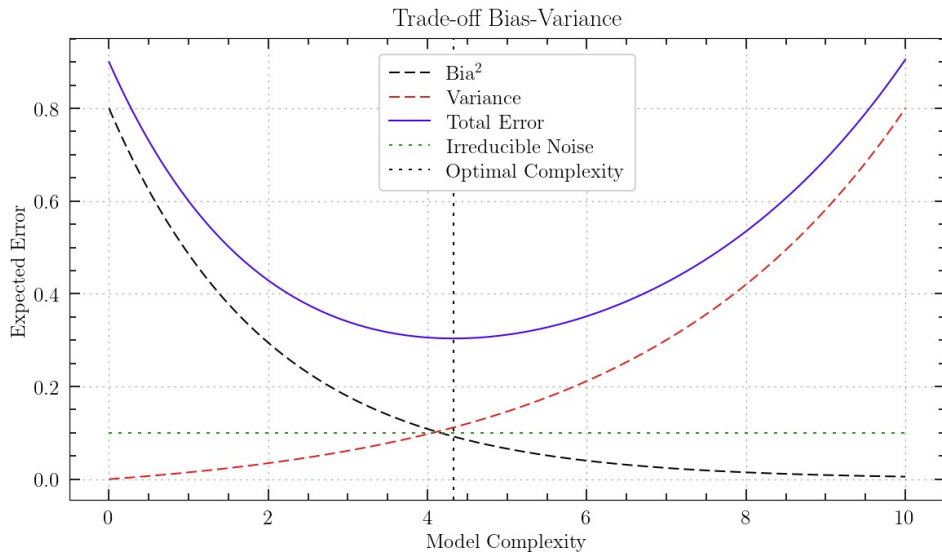
Model Complexity



Model Complexity



Model Complexity





Principal Components Analysis

Principal Components Analysis

Principal Component Analysis

- Dimensionality reduction for high-dimensional data
- Goal: Compact representation with minimal information loss
- **Applications:** compression, latent structure discovery

Intuition Behind PCA

- High-dimensional data is often redundant (overcomplete)
- Example: RGB image (640×480 921, 600D vector)
- PCA seeks a low-dimensional subspace that retains maximum information

Problem Setting

Consider an i.i.d. dataset $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\} \subset \mathbb{R}^D$, with mean $\mathbf{0}$ that possesses the *data covariance matrix*:

$$\mathbf{S} = \frac{1}{N} \sum_{i=1}^N \mathbf{x}_i \mathbf{x}_i^T.$$

Furthermore, we assume there exists a low-dimensional compressed representation (code) $\mathbf{z}_i = \mathbf{B}^T \mathbf{x}_i \in \mathbb{R}^M$ of $\mathbf{x}_i$ where we define the projection matrix:

$$\mathbf{B} := [\mathbf{b}_1, \dots, \mathbf{b}_M] \in \mathbb{R}^{D \times M}.$$

We assume that the columns of $\mathbf{B}$ are orthonormal so that $\mathbf{b}_i^T \mathbf{b}_j = 0$ if and only if $i \neq j$ and $\mathbf{b}_i^T \mathbf{b}_i = 1$. We seek an M -dimensional subspace $U \subseteq \mathbb{R}^D$, $\dim(U) = M < D$ onto which we project the data.

Problem Setting

We denote the projected data by $\tilde{\mathbf{x}}_n \in U$, and their coordinates (with respect to the basis vectors $\mathbf{b}_1, \dots, \mathbf{b}_M$ of U) by $\mathbf{z}_n$. Our aim is to find projections $\tilde{\mathbf{x}}_n \in \mathbb{R}^D$ (or equivalently the codes $\mathbf{z}_n$ and the basis vectors $\mathbf{b}_1, \dots, \mathbf{b}_M$) so that they are as similar to the original data $\mathbf{x}_n$ and minimize the loss due to compression.

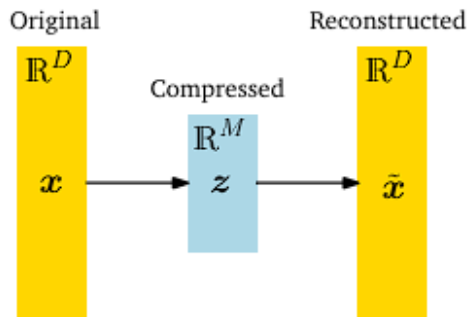
Problem Setting

1. Given data $\mathcal{X} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\} \subset \mathbb{R}^D$.
2. Want to low-dimensional code
 $\mathbf{z}_n = \mathbf{B}^T \mathbf{x}_n \in \mathbb{R}^M$.
3. Projection matrix
 $\mathbf{B} = [\mathbf{b}_1, \dots, \mathbf{b}_M] \in \mathbb{R}^{D \times M}$.

Objective in PCA

1. Find $\mathbf{z}_n$ such that $\tilde{\mathbf{x}}_n = \mathbf{B}\mathbf{B}^T \mathbf{x}_n$ is close to $\mathbf{x}_n$.
2. $\mathbf{B}$ should have orthonormal columns.
3. Compression $\implies$ fewer parameters, less storage

PCA as an Autoencoder



- In PCA, we find a compressed version $\mathbf{z}$ of original data $\mathbf{x}$. The compressed data can be reconstructed into $\tilde{\mathbf{x}}$, which lives in the original data space, but has an intrinsic lower-dimensional representation than $\mathbf{x}$.

Problem Setting

Example: Coordinate Representation/Code

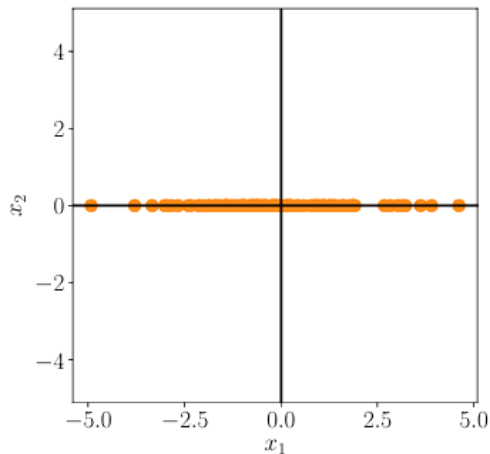
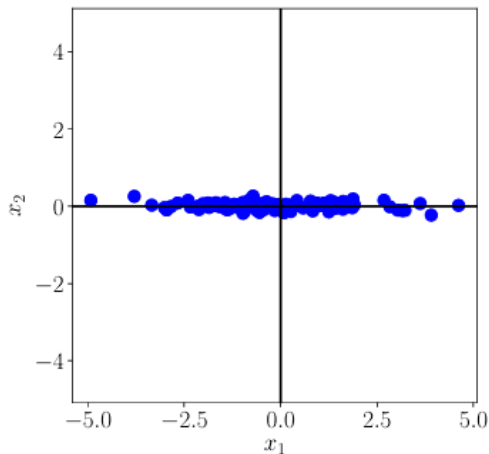
Consider $\mathbb{R}^2$ with the canonical basis $\mathbf{e}_1 = [1, 0]^T$, $\mathbf{e}_2 = [0, 1]^T$. From Linear Algebra notes, we know that $\mathbf{x} \in \mathbb{R}^2$ can be represented as a linear combination of these basis vectors, e.g.:

$$\begin{bmatrix} 5 \\ 3 \end{bmatrix} = 5\mathbf{e}_1 + 3\mathbf{e}_2 \implies \tilde{\mathbf{x}} = \begin{bmatrix} 0 \\ z \end{bmatrix} \in \mathbb{R}^2, \quad z \in \mathbb{R},$$

they can always be written as $0\mathbf{e}_1 + z\mathbf{e}_2$. To represent these vectors it is sufficient to remember/store the *coordinate/code* z of $\tilde{\mathbf{x}}$ with respect to the $\mathbf{e}_2$ vector.

More precisely, the set of $\tilde{\mathbf{x}}$ vectors (with the standard vector addition and scalar multiplication) forms a vector subspace U with $\dim(U) = 1$ because $U = \text{span}[\mathbf{e}_2]$.

Problem Setting



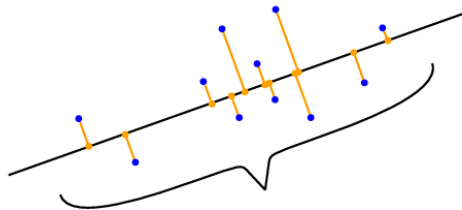
- **Left:** Dataset with x_1 and x_2 coordinates.
- **Right:** Compressed dataset where only the x_1 coordinate is relevant.

Maximum Variance Perspective

If we interpret information content in the data as how “space filling” the dataset is, then we can describe the information contained in the data by looking at the spread of the data.

We know that the variance is an indicator of the spread of the data, and we can derive PCA as a dimensionality reduction algorithm that maximizes the variance in the low-dimensional representation of the data to retain as much information as possible.

- **Core idea:** maximize spread (variance) of the projected data
- Variance reflects information content
- Projection directions = directions of largest variance

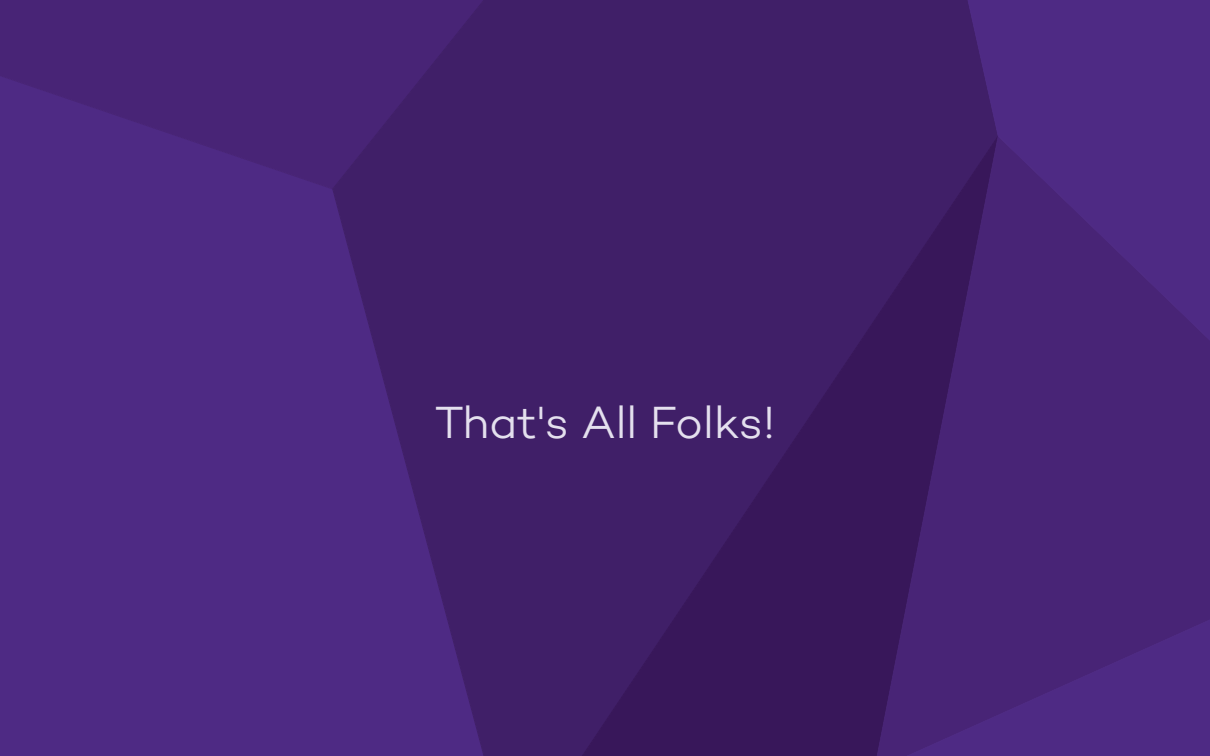


Direction with Maximal Variance

We maximize the variance of the low-dimensional code using a sequential approach. We start by seeking a single vector $\mathbf{b}_1 \in \mathbb{R}^D$ that maximizes the variance of the projected data, i.e., we aim to maximize the variance of the first coordinate z_1 of $\mathbf{z} \in \mathbb{R}^M$ so that:

$$V_1 := \mathbb{V}[z_1] = \frac{1}{N} \sum_{n=1}^N z_{1n}^2,$$

is maximized, where we exploited the i.i.d. assumption of the data and defined z_{1n} as the first coordinate of the low-dimensional representation $\mathbf{z}_n \in \mathbb{R}^M$ of $\mathbf{x}_n \in \mathbb{R}^D$. Note that first component of $\mathbf{z}_n$ is given by $z_{1n} = \mathbf{b}_1^T \mathbf{x}_n$

The background is composed of several overlapping, semi-transparent purple polygons of various shapes and sizes, creating a layered, geometric effect. The colors range from a deep, dark purple to a slightly lighter, more vibrant shade.

That's All Folks!